

CS691 – DBMS LAB

Complete Solved Lab Manual

Oracle SQL & PL/SQL Solutions for every assignment (2–12)
+ Oracle SQL & PL/SQL Syntax Guide

Contents

1. Part I — Oracle SQL & PL/SQL Syntax Guide
2. Assignment 2 — DDL / DML / DQL on CUSTOMER table (32 Qs)
3. Assignment 3 — Constraints, Self-Referencing FK (DEPT, CUST_100)
4. Assignment 4 — SAILOR / BOAT / RESERVE — Queries + Functions (73 Qs)
5. Assignment 5 — COMPANY Schema — Joins, Subqueries, Aggregates (20 Qs)
6. Assignment 6 — Sequences & Views (TOUR)
7. Assignment 7 — Practice: Customer / Item / Sale Schema
8. Assignment 8 — PL/SQL Basics: Loops, Conditionals (12 Qs)
9. Assignment 9 — Exception Handling (3 Qs)
10. Assignment 10 — Procedures & Functions (9 Qs)
11. Assignment 11 — Database Triggers (2 Qs)
12. Assignment 12 — Cursors (4 Qs)
13. Practice Assignment — Library Schema (Student/Book/Iss_rec)

Every table the original assignment sheet gave you as data (sailor, boat, reserve, tour, dept, loc_table, etc.) is reproduced here as an actual grid, right next to the SQL that builds it. Where the sheet only specified column names with no sample values (customer, item/sale, library), a representative dataset is shown the same way so every query below returns a real, checkable result.

Oracle SQL & PL/SQL Syntax Guide

This assumes you already know standard SQL. It is a plain reference for how Oracle spells things and how PL/SQL is written — no other-database comparison, just the commands.

1. DDL — Defining & Changing Tables

Command / Concept	Syntax & Example
CREATE TABLE	CREATE TABLE t (col1 datatype [constraint], col2 datatype, ...);
ALTER TABLE - add column	ALTER TABLE t ADD (new_col datatype);
ALTER TABLE - modify column	ALTER TABLE t MODIFY (col newdatatype);
ALTER TABLE - drop column	ALTER TABLE t DROP COLUMN col;
Rename a table	RENAME old_name TO new_name;
Rename a column	ALTER TABLE t RENAME COLUMN old_col TO new_col;
Truncate (remove rows, keep structure)	TRUNCATE TABLE t;
Drop a table	DROP TABLE t;
Drop permanently (bypass recycle bin)	DROP TABLE t PURGE;
Undo an accidental DROP	FLASHBACK TABLE t TO BEFORE DROP;
Copy structure + data (CTAS)	CREATE TABLE t2 AS SELECT * FROM t1;
Copy structure only (no rows)	CREATE TABLE t2 AS SELECT * FROM t1 WHERE 1 = 2;

2. Constraints

Command / Concept	Syntax & Example
NOT NULL (column level)	col datatype NOT NULL
UNIQUE (column level)	col datatype UNIQUE
UNIQUE (table level)	CONSTRAINT uq_name UNIQUE (col1, col2)
PRIMARY KEY (column level)	col datatype PRIMARY KEY
PRIMARY KEY (table level / composite)	CONSTRAINT pk_name PRIMARY KEY (col1, col2)
FOREIGN KEY (column level)	col datatype REFERENCES other_table(col)
FOREIGN KEY (table level)	CONSTRAINT fk_name FOREIGN KEY (col) REFERENCES other_table(col)
CHECK (column level)	col datatype CHECK (col > 0)
CHECK (table level)	CONSTRAINT ck_name CHECK (salary BETWEEN 4000 AND 25000)
Add a constraint after creation	ALTER TABLE t ADD CONSTRAINT fk_name FOREIGN KEY (col) REFERENCES t2(col);

3. DML — Inserting, Updating, Deleting

Command / Concept	Syntax & Example
Insert a row	INSERT INTO t VALUES (v1, v2, ...);
Insert into named columns	INSERT INTO t (col1, col2) VALUES (v1, v2);
Insert rows from another table	INSERT INTO t2 SELECT * FROM t1 WHERE condition;

Command / Concept	Syntax & Example
Update rows	UPDATE t SET col = value WHERE condition;
Delete rows	DELETE FROM t WHERE condition;
Upsert (merge)	MERGE INTO t USING src ON (t.id = src.id) WHEN MATCHED THEN UPDATE SET t.col = src.col WHEN NOT MATCHED THEN INSERT (id,col) VALUES (src.id, src.col);

4. DQL — Querying

Command / Concept	Syntax & Example
Basic select	SELECT col1, col2 FROM t;
Filter rows	SELECT * FROM t WHERE condition;
Remove duplicates	SELECT DISTINCT col FROM t;
Sort results	SELECT * FROM t ORDER BY col1 ASC, col2 DESC;
Group + aggregate	SELECT col, COUNT(*) FROM t GROUP BY col;
Filter groups	SELECT col, COUNT(*) FROM t GROUP BY col HAVING COUNT(*) > 1;
Inner join	SELECT * FROM a JOIN b ON a.id = b.id;
Outer join (ANSI)	SELECT * FROM a LEFT JOIN b ON a.id = b.id;
Old-style outer join (legacy Oracle)	SELECT * FROM a, b WHERE a.id = b.id(+); -- (+) marks the optional side
Subquery - single value	SELECT * FROM t WHERE col = (SELECT MAX(col) FROM t);
Subquery - multiple values	SELECT * FROM t WHERE col IN (SELECT col FROM t2);
Subquery with ANY / ALL	SELECT * FROM t WHERE col > ANY (SELECT col FROM t2);
EXISTS	SELECT * FROM t WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.id = t.id);
Set operators	SELECT ... UNION UNION ALL INTERSECT MINUS SELECT ...;
Limit rows (pre-12c)	SELECT * FROM t WHERE ROWNUM <= 5;
Limit rows (12c+)	SELECT * FROM t FETCH FIRST 5 ROWS ONLY;
Skip + limit / pagination (12c+)	SELECT * FROM t OFFSET 5 ROWS FETCH NEXT 5 ROWS ONLY;
One-row dummy table	SELECT SYSDATE FROM DUAL;

5. DCL & TCL — Privileges & Transactions

Command / Concept	Syntax & Example
Grant privilege	GRANT SELECT, INSERT ON t TO username;
Revoke privilege	REVOKE SELECT, INSERT ON t FROM username;
Commit a transaction	COMMIT;
Undo a transaction	ROLLBACK;
Partial rollback	SAVEPOINT sp1; ... ROLLBACK TO sp1;

6. Sequences & Views

Command / Concept	Syntax & Example
Create a sequence	CREATE SEQUENCE seq_name START WITH 1 INCREMENT BY 1;
Get next value	seq_name.NEXTVAL -- e.g. INSERT INTO t VALUES (seq_name.NEXTVAL, ...);
Get current value (after a NEXTVAL this session)	seq_name.CURRVAL
Create a view	CREATE [OR REPLACE] VIEW v AS SELECT ... FROM t WHERE ...;
Block inserts that violate the view's WHERE	CREATE VIEW v AS SELECT ... WITH CHECK OPTION;
Make a view non-editable	CREATE VIEW v AS SELECT ... WITH READ ONLY;
Drop a view	DROP VIEW v;

7. Numeric Functions

Command / Concept	Syntax & Example
ABS(n)	Absolute value -> ABS(-5) = 5
POWER(n, p)	n raised to power p -> POWER(2,3) = 8
SQRT(n)	Square root -> SQRT(16) = 4
ROUND(n, d)	Round to d decimals -> ROUND(15.678, 1) = 15.7
TRUNC(n, d)	Truncate, no rounding -> TRUNC(15.678, 1) = 15.6
FLOOR(n)	Largest integer <= n -> FLOOR(15.9) = 15
CEIL(n)	Smallest integer >= n -> CEIL(15.1) = 16
MOD(n, m)	Remainder -> MOD(10,3) = 1
GREATEST(a,b,...)	Largest of the list -> GREATEST(3,7,2) = 7
LEAST(a,b,...)	Smallest of the list -> LEAST(3,7,2) = 2

8. String Functions

Command / Concept	Syntax & Example
LOWER(s) / UPPER(s)	Case conversion -> LOWER('ABC') = 'abc'
INITCAP(s)	Capitalize each word -> INITCAP('john doe') = 'John Doe'
SUBSTR(s, start, len)	Extract substring -> SUBSTR('Database',1,4) = 'Data'
INSTR(s, sub)	Position of substring -> INSTR('Database','base') = 5
LENGTH(s)	Length of string -> LENGTH('Oracle') = 6
LPAD / RPAD(s, n, 'c')	Pad left/right to length n -> LPAD('5',3,'0') = '005'
TRIM(s) / LTRIM(s) / RTRIM(s)	Remove leading/trailing spaces (or given chars)
Concatenation	'a' 'b' or CONCAT('a','b') (Oracle CONCAT takes only 2 args)
SOUNDEX(s)	Phonetic code, useful for 'sounds like' matching

9. Date Functions

Command / Concept	Syntax & Example
SYSDATE	Current date & time -> SELECT SYSDATE FROM DUAL;
SYSTIMESTAMP	Current timestamp with fractional seconds and time zone
ADD_MONTHS(d, n)	Add n months to date d -> ADD_MONTHS(SYSDATE, 2)
MONTHS_BETWEEN(d1, d2)	Number of months between two dates
LAST_DAY(d)	Last calendar day of d's month
NEXT_DAY(d, 'MONDAY')	Next occurrence of the given weekday after d
ROUND(d,'fmt') / TRUNC(d,'fmt')	Round/truncate a date to a unit, e.g. TRUNC(d,'MM') = 1st of month

10. Conversion Functions

Command / Concept	Syntax & Example
TO_CHAR(value, 'format')	Number/date to text -> TO_CHAR(SYSDATE, 'DD-MON-YYYY')

Command / Concept	Syntax & Example
TO_NUMBER(string)	Text to number -> TO_NUMBER('123') = 123
TO_DATE(string, 'format')	Text to date -> TO_DATE('25-DEC-2024', 'DD-MON-YYYY')

11. PL/SQL — Block Structure & Variables

Command / Concept	Syntax & Example
Anonymous block skeleton	<pre>DECLARE -- variable declarations BEGIN -- statements EXCEPTION -- error handling END; /</pre>
Declare a variable	<pre>v_name VARCHAR2(20); v_count NUMBER := 0;</pre>
Anchor a variable's type to a column	<pre>v_name sailor.sname%TYPE;</pre>
Anchor a variable to an entire row	<pre>v_row sailor%ROWTYPE;</pre>
Assign a value	<pre>v_count := v_count + 1;</pre>
Print output (needs SET SERVEROUTPUT ON)	<pre>DBMS_OUTPUT.PUT_LINE('value = ' v_count);</pre>
Run a query into variables	<pre>SELECT sname, age INTO v_name, v_age FROM sailor WHERE sid = 's22';</pre>

12. PL/SQL — Control Structures (IF, LOOP, WHILE, FOR)

Command / Concept	Syntax & Example
IF / ELSIF / ELSE	<pre>IF cond1 THEN ... ELSIF cond2 THEN ... ELSE ... END IF;</pre>
Simple loop	<pre>LOOP EXIT WHEN condition; -- statements END LOOP;</pre>
WHILE loop	<pre>WHILE condition LOOP -- statements END LOOP;</pre>
Numeric FOR loop	<pre>FOR i IN 1..10 LOOP -- statements END LOOP;</pre>
Reverse FOR loop	<pre>FOR i IN REVERSE 1..10 LOOP -- statements END LOOP;</pre>

13. PL/SQL — Cursors

Command / Concept	Syntax & Example
Implicit cursor attributes	<pre>SQL%FOUND, SQL%NOTFOUND, SQL%ROWCOUNT -- after INSERT/UPDATE/DELETE</pre>
Declare an explicit cursor	<pre>CURSOR c_name IS SELECT col1, col2 FROM t WHERE condition;</pre>
Open / fetch / close manually	<pre>OPEN c_name; FETCH c_name INTO v1, v2; CLOSE c_name;</pre>
Loop until no more rows	<pre>LOOP FETCH c_name INTO v1, v2; EXIT WHEN c_name%NOTFOUND; -- statements END LOOP;</pre>

Command / Concept	Syntax & Example
Cursor FOR loop (auto open/fetch/close)	<pre>FOR rec IN c_name LOOP -- use rec.col1, rec.col2 END LOOP;</pre>

14. PL/SQL — Procedures & Functions

Command / Concept	Syntax & Example
Create a procedure	<pre>CREATE OR REPLACE PROCEDURE p_name (p_arg IN NUMBER) IS BEGIN -- statements END; /</pre>
Parameter modes	IN (default, read-only), OUT (returns a value), IN OUT (both)
Call a procedure	<pre>EXEC p_name(10); -- or: BEGIN p_name(10); END; /</pre>
Create a function	<pre>CREATE OR REPLACE FUNCTION f_name (p_arg IN NUMBER) RETURN NUMBER IS BEGIN RETURN p_arg * 2; END; /</pre>
Call a function	<pre>SELECT f_name(5) FROM DUAL; -- or use inside a PL/SQL expression</pre>
Drop a procedure / function	<pre>DROP PROCEDURE p_name; DROP FUNCTION f_name;</pre>

15. PL/SQL — Exception Handling

Command / Concept	Syntax & Example
NO_DATA_FOUND	Raised when SELECT ... INTO returns zero rows
TOO_MANY_ROWS	Raised when SELECT ... INTO returns more than one row
ZERO_DIVIDE	Raised on division by zero
VALUE_ERROR	Raised on a conversion/truncation/size error
DUP_VAL_ON_INDEX	Raised on a unique-constraint violation
OTHERS	Catch-all handler; use SQLERRM to log unexpected errors
Handle an exception	<pre>EXCEPTION WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('not found'); WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE(SQLERRM);</pre>
Declare + raise a user-defined exception	<pre>e_custom EXCEPTION; ... IF condition THEN RAISE e_custom; END IF; ... EXCEPTION WHEN e_custom THEN ...</pre>
Raise a custom application error	<pre>RAISE_APPLICATION_ERROR(-20001, 'Custom error message');</pre>

16. PL/SQL — Triggers

Command / Concept	Syntax & Example
Create a row-level trigger	<pre>CREATE OR REPLACE TRIGGER trg_name BEFORE INSERT OR UPDATE OR DELETE ON t FOR EACH ROW BEGIN -- statements END; /</pre>
Timing options	BEFORE, AFTER, INSTEAD OF (for views)
Detect which DML fired it	<pre>IF INSERTING THEN ... ELSIF UPDATING THEN ... ELSIF DELETING THEN ... END IF;</pre>

Command / Concept	Syntax & Example
Old / new row values	:OLD.col (before change) :NEW.col (after change)
Drop a trigger	DROP TRIGGER trg_name;
Enable / disable a trigger	ALTER TRIGGER trg_name DISABLE; ALTER TRIGGER trg_name ENABLE;

17. Misc

Command / Concept	Syntax & Example
Single-line comment	-- this is a comment
Multi-line comment	/* this is a comment */
Show DBMS_OUTPUT in SQL*Plus / SQL Developer	SET SERVEROUTPUT ON;
Describe a table's structure	DESC t; or DESCRIBE t;

Assignment 2 — CUSTOMER Table (DDL / DML / DQL)

customer(cust_id, cust_fname, cust_lname, territory, cred_lmt, mngr_id, marital_status, sex, income)

Sample data used below

The assignment sheet lists only the column names, not actual values, so this representative dataset is used for every query so results are checkable. Swap in your own roll-list data if your lab supplies a fixed dataset.

CUST_ID	FNAME	LNAME	TERRITORY	CRED_LMT	MNGR_ID	STATUS	SEX	INCOME
C61	Meg	Sen	America	20000	M100	Married	F	45000
C62	John	Doe	America	25000	M100	Single	M	60000
C63	Anna	Roy	India	15000	M101	Single	F	55000
C64	Rita	Kapoor	India	22000	M101	Married	F	20000
C65	Marco	Rossi	Italy	18000	M102	Single	M	40000
C66	Luca	Bianchi	Italy	5000	M102	Married	M	24000
C67	Tom	Brown	America	7000	M100	Single	M	52000
C68	Sara	Khan	India	3000	M101	Married	F	28000
C69	Priya	Nair	India	25000	M101	Single	F	300
C70	Mia	Verdi	Italy	6000	M102	Single	F	4500

1. Create and insert given data in table customer.

```
CREATE TABLE customer (  
    cust_id VARCHAR2(5) PRIMARY KEY,  
    cust_fname VARCHAR2(15),  
    cust_lname VARCHAR2(15),  
    territory VARCHAR2(15),  
    cred_lmt NUMBER(8,2),  
    mngr_id VARCHAR2(6),  
    marital_status VARCHAR2(10),  
    sex CHAR(1),  
    income NUMBER(10,2)  
);  
  
INSERT INTO customer VALUES ('C61','Meg','Sen','America',20000,'M100','Married','F',45000);  
INSERT INTO customer VALUES ('C62','John','Doe','America',25000,'M100','Single','M',60000);  
INSERT INTO customer VALUES ('C63','Anna','Roy','India',15000,'M101','Single','F',55000);  
INSERT INTO customer VALUES ('C64','Rita','Kapoor','India',22000,'M101','Married','F',20000);  
INSERT INTO customer VALUES ('C65','Marco','Rossi','Italy',18000,'M102','Single','M',40000);  
INSERT INTO customer VALUES ('C66','Luca','Bianchi','Italy',5000,'M102','Married','M',24000);  
INSERT INTO customer VALUES ('C67','Tom','Brown','America',7000,'M100','Single','M',52000);  
INSERT INTO customer VALUES ('C68','Sara','Khan','India',3000,'M101','Married','F',28000);  
INSERT INTO customer VALUES ('C69','Priya','Nair','India',25000,'M101','Single','F',300);  
INSERT INTO customer VALUES ('C70','Mia','Verdi','Italy',6000,'M102','Single','F',4500);  
  
-- Note: the assignment sheet lists only the column names, not actual data values,  
-- so representative sample rows (above) are used so every query below returns a  
-- meaningful, checkable result. Replace with your own roll-list data if your lab  
-- supplies a fixed dataset.
```

2. Alter table and add column stay_from_year.

```
ALTER TABLE customer ADD (stay_from_year NUMBER(4));
```

3. Set value of stay_from_year as 2001 for Italy/America and 2003 otherwise.

```
UPDATE customer SET stay_from_year = 2001 WHERE territory IN ('Italy','America');  
UPDATE customer SET stay_from_year = 2003 WHERE territory NOT IN ('Italy','America');
```

4. Display credit limit attribute for America.

```
SELECT cred_lmt FROM customer WHERE territory = 'America';
```

5. Delete the record corresponding to Meg Sen.

```
DELETE FROM customer WHERE cust_fname = 'Meg' AND cust_lname = 'Sen';
```

6. Show all attributes for Italy (all data in territory Italy).

```
SELECT * FROM customer WHERE territory = 'Italy';
```

7. If territory is India and status is Single, set value of credit to 7000.

```
UPDATE customer SET cred_lmt = 7000 WHERE territory = 'India' AND marital_status = 'Single';
```

8. Rename cust_fname to first_name.

```
ALTER TABLE customer RENAME COLUMN cust_fname TO first_name;
```

9. Rename cust_lname to last_name.

```
ALTER TABLE customer RENAME COLUMN cust_lname TO last_name;
```

10. Create table cust1 from the old table customer (copy structure + data, CTAS).

```
CREATE TABLE cust1 AS SELECT * FROM customer;
```

11. Create table cust2 without values of cust1 (structure only, CTAS).

```
CREATE TABLE cust2 AS SELECT * FROM cust1 WHERE 1 = 2;
```

12. Create table cust3 with cust_id, cust_fname, cust_lname, income from old customer, only 5 rows (CTAS).

```
CREATE TABLE cust3 AS SELECT cust_id, first_name, last_name, income FROM customer WHERE ROWNUM <= 5;
```

13. Create table cust4 with customer_id, firstname, lastname, income from old customer (CTAS, renamed columns).

```
CREATE TABLE cust4 AS SELECT cust_id AS customer_id, first_name AS firstname, last_name AS lastname, income FROM customer;
```

14. Drop column income from cust1.

```
ALTER TABLE cust1 DROP COLUMN income;
```

15. Rename table cust1 to cust_one.

```
RENAME cust1 TO cust_one;
```

16. Insert values into cust2 table from customer table.

```
INSERT INTO cust2 SELECT * FROM customer;
```

17. Insert into cust3 (cust_id, f_name, l_name, income) from customer where income > 50000.

```
INSERT INTO cust3 (cust_id, cust_fname, cust_lname, income)
SELECT cust_id, first_name, last_name, income FROM customer WHERE income > 50000;
```

18. Alter table cust4: change customer_id to VARCHAR2(6) and income to NUMBER(5).

```
ALTER TABLE cust4 MODIFY (customer_id VARCHAR2(6));
ALTER TABLE cust4 MODIFY (income NUMBER(5));
```

19. Add new attribute mngr_name to cust4 and insert 5 records.

```
ALTER TABLE cust4 ADD (mngr_name VARCHAR2(15));
UPDATE cust4 SET mngr_name = 'Ravi Sharma' WHERE ROWNUM <= 1;
-- repeat similar single-row UPDATES (or re-INSERT 5 fresh rows including mngr_name)
-- for the remaining records, e.g.:
INSERT INTO cust4 (customer_id, firstname, lastname, income, mngr_name)
VALUES ('C90', 'Neha', 'Verma', 30000, 'Ravi Sharma');
```

20. Add attribute territory to cust4.

```
ALTER TABLE cust4 ADD (territory VARCHAR2(15));
```

21. Drop table cust3 and then bring it back.

```
DROP TABLE cust3;
FLASHBACK TABLE cust3 TO BEFORE DROP;
```

22. Increase the size of the column cust_id by 5.

```
ALTER TABLE customer MODIFY (cust_id VARCHAR2(10)); -- (5 original + 5 = 10)
```

23-24. Customer C63 has changed her last name to be same as customer C68; update accordingly.

```
UPDATE customer SET last_name = (SELECT last_name FROM customer WHERE cust_id = 'C68') WHERE
cust_id = 'C63';
```

25. Display the records where territory = America and cred_lmt = 25000.

```
SELECT * FROM customer WHERE territory = 'America' AND cred_lmt = 25000;
```

26. Display the records of all Indian customers whose income > 20000.

```
SELECT * FROM customer WHERE territory = 'India' AND income > 20000;
```

27. Display the name of the customer having cred_lmt between 2000 and 7000.

```
SELECT first_name, last_name FROM customer WHERE cred_lmt BETWEEN 2000 AND 7000;
```

28. Display records of customers having income 20000, 24000, 300, 4500 using one query.

```
SELECT * FROM customer WHERE income IN (20000, 24000, 300, 4500);
```

29. Display the records in ascending order of first name.

```
SELECT * FROM customer ORDER BY first_name ASC;
```

30. Display the records in descending order of income.

```
SELECT * FROM customer ORDER BY income DESC;
```

31. Insert a duplicate record and display all the records.

```
INSERT INTO customer SELECT * FROM customer WHERE cust_id = 'C61';  
-- (note: cust_id is a PRIMARY KEY, so a true duplicate PK will raise an error;  
-- either drop the PK constraint temporarily or duplicate a table without a PK)  
SELECT * FROM customer;
```

32. Select distinct names of customers.

```
SELECT DISTINCT first_name, last_name FROM customer;
```

Assignment 3 — Constraints & Self-Referencing Foreign Key

DEPT — structure

Column name	Data type(size)	Constraints
dept_id	number(3)	primary key
dept_name	varchar2(15)	-----

DEPT — data to insert

DEPT_ID	DEPT_NAME
90	Sales
69	HR
100	Finance
110	IT

CUST_100 — structure

Column name	Data type(size)	Constraints
emp_id	number(3)	Primary key
first_name	varchar2(10)	Initial letter capital
last_name	varchar2(10)	Initial letter capital, not null
e_mail	varchar2(20)	All upper case
ph_no	varchar2(15)	-----
hire_date	Date	Should be > 01-jan-1980
job_id	varchar2(10)	Must begin with FI, AD, or IT
salary	number(8,2)	>= 4000 and <= 25000
mgr_id	number(3)	-----
dept_id	number(3)	Foreign key -> dept

Table setup SQL

```

CREATE TABLE dept (
    dept_id NUMBER(3) PRIMARY KEY,
    dept_name VARCHAR2(15)
);

INSERT INTO dept VALUES (90, 'Sales');
INSERT INTO dept VALUES (69, 'HR');
INSERT INTO dept VALUES (100, 'Finance');
INSERT INTO dept VALUES (110, 'IT');

CREATE TABLE cust_100 (
    emp_id NUMBER(3) PRIMARY KEY,
    first_name VARCHAR2(10) CONSTRAINT chk_fn CHECK (first_name = INITCAP(first_name)),
    last_name VARCHAR2(10) NOT NULL CONSTRAINT chk_ln CHECK (last_name = INITCAP(last_name)),
    e_mail VARCHAR2(20) CONSTRAINT chk_email CHECK (e_mail = UPPER(e_mail)),
    ph_no VARCHAR2(15),
    hire_date DATE CONSTRAINT chk_hd CHECK (hire_date > TO_DATE('01-JAN-1980', 'DD-MON-YYYY')),
    job_id VARCHAR2(10) CONSTRAINT chk_job CHECK (job_id LIKE 'FI%' OR job_id LIKE 'AD%' OR
job_id LIKE 'IT%'),
    salary NUMBER(8,2) CONSTRAINT chk_sal CHECK (salary BETWEEN 4000 AND 25000),
    mgr_id NUMBER(3),
    dept_id NUMBER(3) CONSTRAINT fk_dept REFERENCES dept(dept_id)
);

```

1. Add 10 records to cust_100.

```

INSERT INTO cust_100 VALUES (101, 'Ravi', 'Kumar', 'RAVI@CO.COM', '9800000001',
    TO_DATE('12-JUN-2005', 'DD-MON-YYYY'), 'IT01', 12000, NULL, 110);
INSERT INTO cust_100 VALUES (102, 'Sita', 'Roy', 'SITA@CO.COM', '9800000002',
    TO_DATE('05-MAR-2008', 'DD-MON-YYYY'), 'FI01', 15000, 101, 100);
INSERT INTO cust_100 VALUES (103, 'Amit', 'Sen', 'AMIT@CO.COM', '9800000003',
    TO_DATE('19-JUL-2010', 'DD-MON-YYYY'), 'AD01', 9000, 101, 90);
INSERT INTO cust_100 VALUES (104, 'Neha', 'Das', 'NEHA@CO.COM', '9800000004',
    TO_DATE('23-AUG-2011', 'DD-MON-YYYY'), 'IT02', 11000, 101, 110);
INSERT INTO cust_100 VALUES (105, 'Arjun', 'Ghosh', 'ARJUN@CO.COM', '9800000005',
    TO_DATE('14-FEB-2012', 'DD-MON-YYYY'), 'FI02', 13500, 102, 100);
INSERT INTO cust_100 VALUES (106, 'Kabir', 'Nath', 'KABIR@CO.COM', '9800000006',
    TO_DATE('01-JAN-2015', 'DD-MON-YYYY'), 'AD02', 8000, 105, 90);
INSERT INTO cust_100 VALUES (107, 'Meera', 'Iyer', 'MEERA@CO.COM', '9800000007',
    TO_DATE('11-NOV-2016', 'DD-MON-YYYY'), 'IT03', 10500, 105, 110);
INSERT INTO cust_100 VALUES (108, 'Rohan', 'Basu', 'ROHAN@CO.COM', '9800000008',
    TO_DATE('30-APR-2017', 'DD-MON-YYYY'), 'FI03', 14000, 105, 100);
INSERT INTO cust_100 VALUES (109, 'Divya', 'Rao', 'DIVYA@CO.COM', '9800000009',
    TO_DATE('09-SEP-2019', 'DD-MON-YYYY'), 'AD03', 7500, 109, 90);
INSERT INTO cust_100 VALUES (110, 'Karan', 'Malhotra', 'KARAN@CO.COM', '9800000010',
    TO_DATE('17-DEC-2020', 'DD-MON-YYYY'), 'IT04', 12500, 109, 110);

```

2. Drop column mgr_id.

```

ALTER TABLE cust_100 DROP COLUMN mgr_id;

```

3. Add column mgr_id back, self-referenced: first 4 report to 1st emp_id, next 4 to 5th emp_id, last 2 to 9th emp_id.

```
ALTER TABLE cust_100 ADD (mgr_id NUMBER(3));
ALTER TABLE cust_100 ADD CONSTRAINT fk_selfmgr FOREIGN KEY (mgr_id) REFERENCES cust_100(emp_id);

UPDATE cust_100 SET mgr_id = 101 WHERE emp_id IN (102,103,104,105);
UPDATE cust_100 SET mgr_id = 105 WHERE emp_id IN (106,107,108,109);
UPDATE cust_100 SET mgr_id = 109 WHERE emp_id IN (110,101);
-- emp_id 101 is the top manager itself and can be left NULL instead of self-referencing:
UPDATE cust_100 SET mgr_id = NULL WHERE emp_id = 101;
```

Assignment 4 — SAILOR / BOAT / RESERVE

SAILOR — structure

Attribute	Datatype	Constraints
SID	varchar2(4)	primary key and start with small s
SNAME	varchar2(15)	initial letter capital
MNAME	varchar2(15)	-----
SURNAME	varchar2(15)	not null
RATING	number(2)	default zero
AGE	number(3,1)	not null

SAILOR — given data

SID	SNAME	MNAME	SURNAME	RATING	AGE
s22	Fredrico		Roberts	7	45
s31	Lubber		Sheen	8	55.5
s32	Charlotte		Gordin	8	25.5
s58	Mary	Beth	Lyon	10	35
s64	Horatio		Powell	7	35.4
s71	Zorba		Alex	10	16
s29	Brutus		Slater	1	33.8
s95	Deep	Graceb	Davis	3	63.5
s74	Horatio		Forrest	9	35
s85	Sara	Art	Powell	3	25.5
s80	Deep	Kumar	Kumar	6	17
s87	Deep	Kumar	Jha	8	51

BOAT — structure

Attribute	Datatype	Constraints
BID	number(3)	primary key & start with small b
BNAME	varchar2(10)	all upper case
COLOR	varchar2(6)	red, green, blue

BOAT — given data

BID	BNAME	COLOR
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

RESERVE — structure

Attribute	Datatype	Constraints
SID	varchar2(4)	Foreign key referencing sailor
BID	number(3)	Foreign key referencing boat
DAY	date	< 1-JAN-2000
SID, BID	--	composite primary key

RESERVE — given data

SID	BID	R_DAY
s22	101	10-OCT-98
s22	103	10-AUG-98
s22	102	10-OCT-98
s22	104	10-JUL-98
s31	102	11-OCT-98
s31	102	11-JUN-98
s31	104	11-DEC-98
s64	101	09-MAY-98
s64	102	09-AUG-98
s74	103	09-AUG-98
s80	102	07-JUL-98
s87	101	08-JUL-98
s87	102	12-DEC-98

Table setup SQL

```

CREATE TABLE sailor (
    sid VARCHAR2(4) PRIMARY KEY CONSTRAINT chk_sid CHECK (sid LIKE 's%'),
    sname VARCHAR2(15) CONSTRAINT chk_sname CHECK (sname = INITCAP(sname)),
    mname VARCHAR2(15),
    surname VARCHAR2(15) NOT NULL,
    rating NUMBER(2) DEFAULT 0,
    age NUMBER(3,1) NOT NULL
);

CREATE TABLE boat (
    bid NUMBER(3) PRIMARY KEY CONSTRAINT chk_bid CHECK (bid > 100),
    bname VARCHAR2(10) CONSTRAINT chk_bname CHECK (bname = UPPER(bname)),
    color VARCHAR2(6) CONSTRAINT chk_color CHECK (color IN ('red','green','blue'))
);

CREATE TABLE reserve (
    sid VARCHAR2(4) REFERENCES sailor(sid),
    bid NUMBER(3) REFERENCES boat(bid),
    r_day DATE CONSTRAINT chk_rday CHECK (r_day < TO_DATE('01-JAN-2000','DD-MON-YYYY')),
    PRIMARY KEY (sid, bid)
);

-- sailor rows
INSERT INTO sailor VALUES ('s22','Fredrico',NULL,'Roberts',7,45);
INSERT INTO sailor VALUES ('s31','Lubber',NULL,'Sheen',8,55.5);
INSERT INTO sailor VALUES ('s32','Charlotte',NULL,'Gordin',8,25.5);
INSERT INTO sailor VALUES ('s58','Mary','Beth','Lyon',10,35);
INSERT INTO sailor VALUES ('s64','Horatio',NULL,'Powell',7,35.4);
INSERT INTO sailor VALUES ('s71','Zorba',NULL,'Alex',10,16);
INSERT INTO sailor VALUES ('s29','Brutus',NULL,'Slater',1,33.8);
INSERT INTO sailor VALUES ('s95','Deep','Graceb','Davis',3,63.5);
INSERT INTO sailor VALUES ('s74','Horatio',NULL,'Forrest',9,35);
INSERT INTO sailor VALUES ('s85','Sara','Art','Powell',3,25.5);
INSERT INTO sailor VALUES ('s80','Deep','Kumar','Kumar',6,17);
INSERT INTO sailor VALUES ('s87','Deep','Kumar','Jha',8,51);

-- boat rows
INSERT INTO boat VALUES (101,'INTERLAKE','blue');
INSERT INTO boat VALUES (102,'INTERLAKE','red');
INSERT INTO boat VALUES (103,'CLIPPER','green');
INSERT INTO boat VALUES (104,'MARINE','red');

-- reserve rows
INSERT INTO reserve VALUES ('s22',101,TO_DATE('10-OCT-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s22',103,TO_DATE('10-AUG-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s22',102,TO_DATE('10-OCT-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s22',104,TO_DATE('10-JUL-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s31',102,TO_DATE('11-OCT-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s31',102,TO_DATE('11-JUN-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s31',104,TO_DATE('11-DEC-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s64',101,TO_DATE('09-MAY-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s64',102,TO_DATE('09-AUG-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s74',103,TO_DATE('09-AUG-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s80',102,TO_DATE('07-JUL-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s87',101,TO_DATE('08-JUL-98','DD-MON-RR'));
INSERT INTO reserve VALUES ('s87',102,TO_DATE('12-DEC-98','DD-MON-RR'));

-- Note on question 11 in the "reserve" data: 's31',102 appears twice on different
-- dates in the sheet -- since (sid,bid) is the composite primary key, only ONE such
-- row can physically exist. Keep whichever date your instructor's data sheet
-- specifies, or drop the composite PK to allow both historical reservation dates.

```

Part A — Queries (1–42)

1. Find the names and ages of all sailors.

```
SELECT sname, age FROM sailor;
```

2. Show names as names_of_sailors and add 2 to age.

```
SELECT sname AS names_of_sailors, age + 2 AS age FROM sailor;
```

3. Select all records from sailors in ascending order by name.

```
SELECT * FROM sailor ORDER BY sname ASC;
```

4. Show all sailors' names.

```
SELECT sname FROM sailor;
```

5. Select all distinct sailors' names.

```
SELECT DISTINCT sname FROM sailor;
```

6. Show all distinct sailors' names and ratings, rating between 5 and 10.

```
SELECT DISTINCT sname, rating FROM sailor WHERE rating BETWEEN 5 AND 10;
```

7. Select all records ascending by rating and descending by age.

```
SELECT * FROM sailor ORDER BY rating ASC, age DESC;
```

8. Select all records from sailors whose rating > 7.

```
SELECT * FROM sailor WHERE rating > 7;
```

9. Find records for sailor name Horatio and age = 35.4.

```
SELECT * FROM sailor WHERE sname = 'Horatio' AND age = 35.4;
```

10. Find records for sailor name Horatio or age = 35.4.

```
SELECT * FROM sailor WHERE sname = 'Horatio' OR age = 35.4;
```

11. Select names of sailors who have reserved boat 104.

```
SELECT s.sname FROM sailor s JOIN reserve r ON s.sid = r.sid WHERE r.bid = 104;
```

12. Find sid of sailors who have reserved a red boat.

```
SELECT DISTINCT r.sid FROM reserve r JOIN boat b ON r.bid = b.bid WHERE b.color = 'red';
```

13. Select records for name beginning with 'B'.

```
SELECT * FROM sailor WHERE sname LIKE 'B%';
```

14. Select records for name containing 'B'/'b'.

```
SELECT * FROM sailor WHERE UPPER(sname) LIKE '%B%';
```

15. Select names where rating is present (not null).

```
SELECT sname FROM sailor WHERE rating IS NOT NULL;
```

16. Select names where rating is absent (null).

```
SELECT sname FROM sailor WHERE rating IS NULL;
```

17. Find color of boats reserved by Lubber.

```
SELECT DISTINCT b.color FROM boat b JOIN reserve r ON b.bid = r.bid  
JOIN sailor s ON s.sid = r.sid WHERE s.sname = 'Lubber';
```

18. Find a sailor name that has reserved at least one boat.

```
SELECT DISTINCT s.sname FROM sailor s WHERE s.sid IN (SELECT sid FROM reserve);
```

19. (renumbered from sheet's 20) Compute the increment of rating for sailors who sailed on different boats on the same day.

```
SELECT r1.sid, s.rating, s.rating + 1 AS incremented_rating  
FROM reserve r1 JOIN reserve r2  
ON r1.sid = r2.sid AND r1.r_day = r2.r_day AND r1.bid <> r2.bid  
JOIN sailor s ON s.sid = r1.sid;
```

20. Find names of sailors whose name begins and ends with B and has at least 3 characters.

```
SELECT sname FROM sailor WHERE sname LIKE 'B%B' AND LENGTH(sname) >= 3;
```

21. Find names of sailors whose name begins and ends with 'B' and has exactly 3 chars.

```
SELECT sname FROM sailor WHERE sname LIKE 'B_B' AND LENGTH(sname) = 3;
```

22. Find names of sailors who have reserved a red boat or a green boat.

```
SELECT DISTINCT s.sname FROM sailor s JOIN reserve r ON s.sid = r.sid  
JOIN boat b ON r.bid = b.bid WHERE b.color IN ('red', 'green');
```

23. Find names of sailors who have reserved a red boat but not a green boat.

```
SELECT DISTINCT s.sname FROM sailor s JOIN reserve r ON s.sid = r.sid
JOIN boat b ON r.bid = b.bid WHERE b.color = 'red'
AND s.sid NOT IN (
    SELECT r2.sid FROM reserve r2 JOIN boat b2 ON r2.bid = b2.bid WHERE b2.color = 'green'
);
```

24. Find names of sailors who have reserved boat 103.

```
SELECT s.sname FROM sailor s JOIN reserve r ON s.sid = r.sid WHERE r.bid = 103;
```

25. Find names of sailors who have reserved a red boat. (repeat)

```
SELECT DISTINCT s.sname FROM sailor s JOIN reserve r ON s.sid = r.sid
JOIN boat b ON r.bid = b.bid WHERE b.color = 'red';
```

26. Find names of sailors who have NOT reserved a red boat.

```
SELECT sname FROM sailor WHERE sid NOT IN (
    SELECT r.sid FROM reserve r JOIN boat b ON r.bid = b.bid WHERE b.color = 'red'
);
```

27. Count distinct sailor names.

```
SELECT COUNT(DISTINCT sname) FROM sailor;
```

28a. Records with rating > SOME rating of sailors named Horatio.

```
SELECT * FROM sailor WHERE rating > SOME (SELECT rating FROM sailor WHERE sname = 'Horatio');
```

28b. Records with rating > ALL ratings of sailors named Horatio.

```
SELECT * FROM sailor WHERE rating > ALL (SELECT rating FROM sailor WHERE sname = 'Horatio');
```

29a. Records with rating < SOME rating of sailors named Horatio.

```
SELECT * FROM sailor WHERE rating < SOME (SELECT rating FROM sailor WHERE sname = 'Horatio');
```

29b. Records with rating < ALL ratings of sailors named Horatio.

```
SELECT * FROM sailor WHERE rating < ALL (SELECT rating FROM sailor WHERE sname = 'Horatio');
```

30. Select all records for sname neither Lubber nor Horatio.

```
SELECT * FROM sailor WHERE sname NOT IN ('Lubber', 'Horatio');
```

31. Find names of sailors whose rating > 10 or 20 or 30 (multirow subquery, ANY).

```
SELECT sname FROM sailor WHERE rating > ANY (SELECT column_value FROM
TABLE(SYS.ODCINUMBERLIST(10,20,30)));
-- simpler equivalent:
SELECT sname FROM sailor WHERE rating > 10 OR rating > 20 OR rating > 30;
```

32. Find names of sailors whose rating > 10 and 20 and 30 (multirow subquery, ALL).

```
SELECT sname FROM sailor WHERE rating > ALL (SELECT column_value FROM
TABLE(SYS.ODCINUMBERLIST(10,20,30)));
-- simpler equivalent:
SELECT sname FROM sailor WHERE rating > 30;
```

33. Find average age of sailors with rating 10.

```
SELECT AVG(age) FROM sailor WHERE rating = 10;
```

34. Find name of sailors older than the oldest sailor with rating = 10.

```
SELECT sname FROM sailor WHERE age > (SELECT MAX(age) FROM sailor WHERE rating = 10);
```

35. Find the age of the youngest sailor for each rating level.

```
SELECT rating, MIN(age) AS youngest_age FROM sailor GROUP BY rating;
```

36. Find the name of each sailor eligible to vote (age >= 18), for each rating level.

```
SELECT rating, sname FROM sailor WHERE age >= 18 ORDER BY rating;
```

37. Find the age of the youngest voting-eligible sailor per rating, only ratings with >= 2 such sailors.

```
SELECT rating, MIN(age) AS youngest_age
FROM sailor WHERE age >= 18
GROUP BY rating HAVING COUNT(*) >= 2;
```

38. Find the average age of sailors for each rating level with at least two such sailors.

```
SELECT rating, AVG(age) FROM sailor GROUP BY rating HAVING COUNT(*) >= 2;
```

39. For each red boat, count the number of reservations for that boat.

```
SELECT b.bid, b.bname, COUNT(r.sid) AS reservation_count
FROM boat b JOIN reserve r ON b.bid = r.bid
WHERE b.color = 'red' GROUP BY b.bid, b.bname;
```

40. Find the sailor(s) with the highest rating.

```
SELECT * FROM sailor WHERE rating = (SELECT MAX(rating) FROM sailor);
```

41. Find the rating(s) for which the average age of sailors is minimum, over all ratings.

```
SELECT rating FROM (
    SELECT rating, AVG(age) avg_age FROM sailor GROUP BY rating
) WHERE avg_age = (SELECT MIN(avg_age) FROM (SELECT rating, AVG(age) avg_age FROM sailor GROUP BY
rating));
```

42. Find sailors who have reserved all boats. (division query)

```
SELECT s.sname FROM sailor s
WHERE NOT EXISTS (
    SELECT b.bid FROM boat b
    WHERE NOT EXISTS (
        SELECT 1 FROM reserve r WHERE r.sid = s.sid AND r.bid = b.bid
    )
);
```

Part B — Functions (46–73)

1. Display sname with left-side padding by at least 3 *.

```
SELECT LPAD(sname, LENGTH(sname) + 3, '*') FROM sailor;
```

2. Display length of each name.

```
SELECT sname, LENGTH(sname) FROM sailor;
```

3. Display all sailors' names in uppercase.

```
SELECT UPPER(sname) FROM sailor;
```

4. Display all sailors' names in lowercase.

```
SELECT LOWER(sname) FROM sailor;
```

5. Display all sailors' names in sentence case.

```
SELECT INITCAP(LOWER(sname)) FROM sailor;
```

6. Display 4th to 7th letter of sailors' name.

```
SELECT SUBSTR(sname, 4, 4) FROM sailor;
```

7. Display 4th and 7th letter of sailors' name.

```
SELECT SUBSTR(sname,4,1) || SUBSTR(sname,7,1) FROM sailor;
```

8. Concat sid, sname.

```
SELECT sid || ' - ' || sname AS sid_sname FROM sailor;
```

9. Display square root of rating.

```
SELECT sname, SQRT(rating) FROM sailor;
```

10. Display floor values of all ages.

```
SELECT sname, FLOOR(age) FROM sailor;
```

11. Display ceiling values of all ages.

```
SELECT sname, CEIL(age) FROM sailor;
```

12. Select all sname with the first 2 letters removed.

```
SELECT SUBSTR(sname, 3) FROM sailor;
```

13. List months between today and reservation date.

```
SELECT r_day, MONTHS_BETWEEN(SYSDATE, r_day) AS months_between FROM reserve;
```

14. Select days between today and reservation date.

```
SELECT r_day, TRUNC(SYSDATE) - r_day AS days_between FROM reserve;
```

15. Shift all reservation days later by 2 months.

```
SELECT r_day, ADD_MONTHS(r_day, 2) AS shifted_later FROM reserve;
```

16. Shift all reservation days earlier by 3 months.

```
SELECT r_day, ADD_MONTHS(r_day, -3) AS shifted_earlier FROM reserve;
```

17. After sailing, sailors enjoy the next Monday as a holiday - find that day.

```
SELECT r_day, NEXT_DAY(r_day, 'MONDAY') AS holiday FROM reserve;
```

18. Display 3 * before and after each sname.

```
SELECT '***' || sname || '***' FROM sailor;
```

19. Find the date when sailing ends (last day of that month, as example rule).

```
SELECT r_day, LAST_DAY(r_day) AS sailing_end FROM reserve;
```

20. Display all reservation days.

```
SELECT r_day FROM reserve;
```

21. Find the position of 'Kumar' in the sailors' name.

```
SELECT sname, mname, INSTR(mname, 'Kumar') AS position_in_mname FROM sailor;
```

22. Display all sailors' names ordered by length.

```
SELECT sname FROM sailor ORDER BY LENGTH(sname);
```

23. Display sid, sname and availability of middle name as 'available'/'not available'.

```
SELECT sid, sname,  
       CASE WHEN mname IS NULL THEN 'not available' ELSE 'available' END AS middle_name_status  
FROM sailor;
```

24. Display all reservation days like '22nd March twenty ten' and 12/09/1998.

```
SELECT TO_CHAR(r_day, 'fmDDth "of" Month YYYY') AS long_format,  
       TO_CHAR(r_day, 'MM/DD/YYYY') AS short_format  
FROM reserve;
```

25. Find the day of the week of reservation date.

```
SELECT r_day, TO_CHAR(r_day, 'DAY') AS weekday_name FROM reserve;
```

26. Find the number (index) of the weekday of reservation date.

```
SELECT r_day, TO_CHAR(r_day, 'D') AS weekday_number FROM reserve;
```

27. Find the number of days passed up to the reservation date of that year.

```
SELECT r_day, TO_CHAR(r_day, 'DDD') AS day_of_year FROM reserve;
```

28. Display the number of weeks of the year for the reservation day.

```
SELECT r_day, TO_CHAR(r_day, 'WW') AS week_of_year FROM reserve;
```

Assignment 5 — COMPANY Schema

Schema — tables & attributes (as given)

Table	Attributes
EMPLOYEE	FNAME, MINIT, LNAME, SSN, BDATE, ADDRESS, SEX, SALARY, SUPERSSN, DNO
DEPARTMENT	DNAME, DNUMBER, MGRSSN, MGRSTARTDATE
DEPT_LOCATIONS	DNUMBER, DLOCATION
PROJECT	PNAME, PNUMBER, PLOCATION, DNUM
WORKS_ON	ESSN, PNO, HOURS
DEPENDENT	ESSN, DEPENDENT_NAME, SEX, BDATE, RELATIONSHIP

Table setup SQL

```

CREATE TABLE department (
    dname VARCHAR2(20),
    dnumber NUMBER(3) PRIMARY KEY,
    mgrssn VARCHAR2(9),
    mgrstartdate DATE
);

CREATE TABLE employee (
    fname VARCHAR2(15),
    minit CHAR(1),
    lname VARCHAR2(15),
    ssn VARCHAR2(9) PRIMARY KEY,
    bdate DATE,
    address VARCHAR2(40),
    sex CHAR(1),
    salary NUMBER(10,2),
    superssn VARCHAR2(9),
    dno NUMBER(3) REFERENCES department(dnumber)
);

ALTER TABLE department ADD CONSTRAINT fk_mgr FOREIGN KEY (mgrssn) REFERENCES employee(ssn);

CREATE TABLE dept_locations (
    dnumber NUMBER(3) REFERENCES department(dnumber),
    dlocation VARCHAR2(20),
    PRIMARY KEY (dnumber, dlocation)
);

CREATE TABLE project (
    pname VARCHAR2(20),
    pnumber NUMBER(3) PRIMARY KEY,
    plocation VARCHAR2(20),
    dnum NUMBER(3) REFERENCES department(dnumber)
);

CREATE TABLE works_on (
    essn VARCHAR2(9) REFERENCES employee(ssn),
    pno NUMBER(3) REFERENCES project(pnumber),
    hours NUMBER(4,1),
    PRIMARY KEY (essn, pno)
);

CREATE TABLE dependent (
    essn VARCHAR2(9) REFERENCES employee(ssn),
    dependent_name VARCHAR2(20),
    sex CHAR(1),
    bdate DATE,
    relationship VARCHAR2(15),
    PRIMARY KEY (essn, dependent_name)
);

-- This is the classic Elmasri & Navathe "COMPANY" database. Populate with the
-- standard sample data used in your textbook/practicals (each department,
-- Research/Administration/Headquarters employees, ProductX/Y/Z projects, etc.).

```

1. For every project in 'Stafford', list project no., dept no., manager's last name, address, birth date.

```
SELECT p.pnumber, p.dnum, e.lname, e.address, e.bdate
FROM project p
JOIN department d ON p.dnum = d.dnumber
JOIN employee e ON d.mgrssn = e.ssn
WHERE p.plocation = 'Stafford';
```

2. Retrieve the name of each employee who works on ALL projects controlled by department 5.

```
SELECT e.fname, e.lname
FROM employee e
WHERE NOT EXISTS (
    SELECT p.pnumber FROM project p WHERE p.dnum = 5
    MINUS
    SELECT w.pno FROM works_on w WHERE w.essn = e.ssn
);
```

3. List project numbers for projects that involve employee 'Smith' as worker OR manager of the controlling dept.

```
SELECT DISTINCT p.pnumber
FROM project p
WHERE p.pnumber IN (
    SELECT w.pno FROM works_on w JOIN employee e ON w.essn = e.ssn WHERE e.lname = 'Smith'
)
OR p.dnum IN (
    SELECT d.dnumber FROM department d JOIN employee e ON d.mgrssn = e.ssn WHERE e.lname = 'Smith'
);
```

4. Retrieve names of employees with no dependents.

```
SELECT fname, lname FROM employee
WHERE ssn NOT IN (SELECT essn FROM dependent);
```

5. List names of managers who have at least one dependent.

```
SELECT e.fname, e.lname FROM employee e
WHERE e.ssn IN (SELECT mgrssn FROM department)
AND e.ssn IN (SELECT essn FROM dependent);
```

6. For each employee, retrieve their name and their immediate supervisor's name.

```
SELECT e.fname, e.lname, s.fname AS sup_fname, s.lname AS sup_lname
FROM employee e LEFT JOIN employee s ON e.superssn = s.ssn;
```

7. Show resulting salaries if every employee working on 'ProductX' gets a 10% raise.

```
SELECT e.fname, e.lname, e.salary AS current_salary, e.salary * 1.10 AS new_salary
FROM employee e
JOIN works_on w ON e.ssn = w.essn
JOIN project p ON w.pno = p.pnumber
WHERE p.pname = 'ProductX';
```

8. Retrieve employees and the projects they work on, ordered by dept then by lname, fname.

```
SELECT d.dname, e.lname, e.fname, p.pname
FROM employee e
JOIN department d ON e.dno = d.dnumber
JOIN works_on w ON e.ssn = w.essn
JOIN project p ON w.pno = p.pnumber
ORDER BY d.dname, e.lname, e.fname;
```

9. Retrieve names of all employees who do not have supervisors.

```
SELECT fname, lname FROM employee WHERE superssn IS NULL;
```

10. Retrieve the name of each employee who has a dependent with the same last name.

```
SELECT e.fname, e.lname FROM employee e
JOIN dependent dep ON e.ssn = dep.essn
WHERE e.lname = dep.dependent_name;
-- (note: if dependent_name stores full name rather than just last name,
-- compare with SUBSTR/INSTR on dep.dependent_name instead)
```

11. Retrieve SSNs of employees who work on project numbers 1 and 2.

```
SELECT essn FROM works_on WHERE pno = 1
INTERSECT
SELECT essn FROM works_on WHERE pno = 2;
```

12. Names of employees whose salary is greater than the salary of ALL employees in dept 5.

```
SELECT fname, lname FROM employee
WHERE salary > ALL (SELECT salary FROM employee WHERE dno = 5);
```

13. Sum, max, min, average salary of all employees.

```
SELECT SUM(salary), MAX(salary), MIN(salary), AVG(salary) FROM employee;
```

14. Sum, max, min, average salary of employees in the 'Research' department.

```
SELECT SUM(e.salary), MAX(e.salary), MIN(e.salary), AVG(e.salary)
FROM employee e JOIN department d ON e.dno = d.dnumber
WHERE d.dname = 'Research';
```

15. Retrieve names of employees who have two or more dependents.

```
SELECT e.fname, e.lname
FROM employee e JOIN dependent dep ON e.ssn = dep.essn
GROUP BY e.fname, e.lname HAVING COUNT(*) >= 2;
```

16. Count employees with salary > 40000 per department, only for depts with more than 5 employees.

```
SELECT dno, COUNT(*) AS high_earners
FROM employee
WHERE dno IN (SELECT dno FROM employee GROUP BY dno HAVING COUNT(*) > 5)
AND salary > 40000
GROUP BY dno;
```

17. For each project: project number, name, and number of employees who work on it.

```
SELECT p.pnumber, p.pname, COUNT(w.essn) AS emp_count
FROM project p LEFT JOIN works_on w ON p.pnumber = w.pno
GROUP BY p.pnumber, p.pname;
```

18. For each project on which more than 2 employees work: project number, name, employee count.

```
SELECT p.pnumber, p.pname, COUNT(w.essn) AS emp_count
FROM project p JOIN works_on w ON p.pnumber = w.pno
GROUP BY p.pnumber, p.pname HAVING COUNT(w.essn) > 2;
```

19. For each project: project number, name, and number of dept-5 employees who work on it.

```
SELECT p.pnumber, p.pname, COUNT(w.essn) AS dept5_emp_count
FROM project p JOIN works_on w ON p.pnumber = w.pno
JOIN employee e ON w.essn = e.ssn
WHERE e.dno = 5
GROUP BY p.pnumber, p.pname;
```

20. For each department with more than 5 employees: dept number and count earning > 40000.

```
SELECT dno, COUNT(*) AS high_earner_count
FROM employee
WHERE salary > 40000
GROUP BY dno
HAVING (SELECT COUNT(*) FROM employee e2 WHERE e2.dno = employee.dno) > 5;
```

Assignment 6 — Sequences & Views

Part A — sample data for the sequence-driven table

ID	NAME	LOCATION
108	Ram	Kolkata
107	Kaushik	Naihati
106	Subhendu	Narayanpara
105	Arti	Bhatpara

Part B — TOUR table (given data)

TOUR_ID	TOUR_SPOT	FARE	TYPE
T1	Gangtok	6000	Delux
T2	Puri	2000	General
T3	Nainital	9000	Delux
T4	Andaman	12000	General
T5	Madurai	7000	Delux

Table setup SQL

```
-- Part A: auto-generated ID using a sequence
CREATE SEQUENCE id_seq START WITH 105 INCREMENT BY 1;

CREATE TABLE loc_table (
    id NUMBER(3) PRIMARY KEY,
    name VARCHAR2(20),
    location VARCHAR2(10)
);

INSERT INTO loc_table VALUES (id_seq.NEXTVAL, 'Arti', 'Bhatpara');
INSERT INTO loc_table VALUES (id_seq.NEXTVAL, 'Subhendu', 'Narayanpara');
INSERT INTO loc_table VALUES (id_seq.NEXTVAL, 'Kaushik', 'Naihati');
INSERT INTO loc_table VALUES (id_seq.NEXTVAL, 'Ram', 'Kolkata');
-- (MySQL equivalent: define id column as AUTO_INCREMENT starting at 105
-- via ALTER TABLE loc_table AUTO_INCREMENT = 105; since it has no native SEQUENCE object)

-- Part B: TOUR table
CREATE TABLE tour (
    tour_id VARCHAR2(4) PRIMARY KEY,
    tour_spot VARCHAR2(15),
    fare NUMBER(8,2),
    type VARCHAR2(10)
);

INSERT INTO tour VALUES ('T1','Gangtok',6000,'Delux');
INSERT INTO tour VALUES ('T2','Puri',2000,'General');
INSERT INTO tour VALUES ('T3','Nainital',9000,'Delux');
INSERT INTO tour VALUES ('T4','Andaman',12000,'General');
INSERT INTO tour VALUES ('T5','Madurai',7000,'Delux');
```

1. Create a view TOURVIEW for deluxe-type tours with tour_spot and fare.

```
CREATE VIEW tourview AS
SELECT tour_spot, fare FROM tour WHERE type = 'Delux';
```

2a. Find tour spots with fare > 8000 and type Delux, from the TOUR table.

```
SELECT tour_spot FROM tour WHERE fare > 8000 AND type = 'Delux';
```

2b. Same, from the TOURVIEW view.

```
SELECT tour_spot FROM tourview WHERE fare > 8000;
```

3. Insert three rows into TOURVIEW with location Bangladesh, Delhi, Hyderabad.

```
INSERT INTO tourview (tour_spot, fare) VALUES ('Bangladesh', 5000);
INSERT INTO tourview (tour_spot, fare) VALUES ('Delhi', 4000);
INSERT INTO tourview (tour_spot, fare) VALUES ('Hyderabad', 6000);
-- these inserts succeed but the "type" column defaults to NULL in the base
-- TOUR table since TOURVIEW does not expose/set it.
```

4. Display these records - are they seen in TOURVIEW? Are they seen in TOUR?

```
SELECT * FROM tourview; -- shows all 3 new rows (fare, tour_spot visible)
SELECT * FROM tour; -- also shows the 3 new rows, but with type = NULL,
-- so they will NOT appear if you filter tour by type='Delux'
```

5. Make them seen in TOURVIEW filtered to Delux only (i.e. the view's WHERE condition).

```
UPDATE tour SET type = 'Delux' WHERE tour_spot IN ('Bangladesh','Delhi','Hyderabad');
-- now SELECT * FROM tourview; will include them, since the view's WHERE type='Delux' matches
```

6. Create a view tour_type_num with tour_type and its total count.

```
CREATE VIEW tour_type_num AS
SELECT type AS tour_type, COUNT(*) AS total_count
FROM tour GROUP BY type;
```

7. Try to insert records via tour_type_num - explain the error.

```
INSERT INTO tour_type_num VALUES ('Delux', 10);
-- ORA-01732: data manipulation operation not legal on this view
-- Reason: the view is based on a GROUP BY / aggregate function (COUNT(*)),
-- so Oracle cannot map an inserted row back to a single, unambiguous row of
-- the base TOUR table. Views containing GROUP BY, aggregate functions,
-- DISTINCT, or set operators are inherently non-updatable/non-insertable.
```

Assignment 7 (Practice) — Customer / Item / Sale

Schema (as given)

Table	Attributes
Customer	Cust_id (integer), cust_name (string)
Item	item_id (integer), item_name (string), price (integer)
Sale	bill_no (integer), bill_date (date), cust_id, item_id, qty_sold

a. Create the tables with appropriate integrity constraints.

```
CREATE TABLE cust7 (  
    cust_id NUMBER(4) PRIMARY KEY,  
    cust_name VARCHAR2(20) NOT NULL  
);  
  
CREATE TABLE item (  
    item_id NUMBER(4) PRIMARY KEY,  
    item_name VARCHAR2(20) NOT NULL,  
    price NUMBER(8,2) CHECK (price > 0)  
);  
  
CREATE TABLE sale (  
    bill_no NUMBER(5) PRIMARY KEY,  
    bill_date DATE DEFAULT SYSDATE,  
    cust_id NUMBER(4) REFERENCES cust7(cust_id),  
    item_id NUMBER(4) REFERENCES item(item_id),  
    qty_sold NUMBER(5) CHECK (qty_sold > 0)  
);
```

b. Insert around 10 records in each of the tables.

```

-- customers
INSERT INTO cust7 VALUES (1,'Alok Basu');
INSERT INTO cust7 VALUES (2,'Bina Roy');
INSERT INTO cust7 VALUES (3,'Chandan Ghosh');
INSERT INTO cust7 VALUES (4,'Dipa Sen');
INSERT INTO cust7 VALUES (5,'Amit Kar');
INSERT INTO cust7 VALUES (6,'Rina Dutta');
INSERT INTO cust7 VALUES (7,'Sanjay Paul');
INSERT INTO cust7 VALUES (8,'Tina Saha');
INSERT INTO cust7 VALUES (9,'Uday Nath');
INSERT INTO cust7 VALUES (10,'Vikram Rao');

-- items
INSERT INTO item VALUES (101,'Notebook',60);
INSERT INTO item VALUES (102,'Pen',10);
INSERT INTO item VALUES (103,'Bag',450);
INSERT INTO item VALUES (104,'Bottle',150);
INSERT INTO item VALUES (105,'Umbrella',300);
INSERT INTO item VALUES (106,'Shoes',900);
INSERT INTO item VALUES (107,'Watch',1200);
INSERT INTO item VALUES (108,'Calculator',350);
INSERT INTO item VALUES (109,'Headphones',700);
INSERT INTO item VALUES (110,'Charger',250);

-- sales (using today's date so "current date" queries return rows)
INSERT INTO sale VALUES (1,SYSDATE,1,101,2);
INSERT INTO sale VALUES (2,SYSDATE,2,103,1);
INSERT INTO sale VALUES (3,SYSDATE,3,106,1);
INSERT INTO sale VALUES (4,SYSDATE,1,107,1);
INSERT INTO sale VALUES (5,SYSDATE,4,102,5);
INSERT INTO sale VALUES (6,SYSDATE,5,109,1);
INSERT INTO sale VALUES (7,SYSDATE,5,110,2);
INSERT INTO sale VALUES (8,SYSDATE,6,104,3);
INSERT INTO sale VALUES (9,SYSDATE,7,105,1);
INSERT INTO sale VALUES (10,SYSDATE,8,108,1);

```

c. List all bills for the current date with customer names and item numbers.

```

SELECT s.bill_no, c.cust_name, s.item_id
FROM sale s JOIN cust7 c ON s.cust_id = c.cust_id
WHERE TRUNC(s.bill_date) = TRUNC(SYSDATE);

```

d. List total bill details with quantity sold, price, and final amount.

```

SELECT s.bill_no, s.qty_sold, i.price, (s.qty_sold * i.price) AS final_amount
FROM sale s JOIN item i ON s.item_id = i.item_id;

```

e. List details of customers who bought a product with price > 200.

```

SELECT DISTINCT c.* FROM cust7 c
JOIN sale s ON c.cust_id = s.cust_id
JOIN item i ON s.item_id = i.item_id
WHERE i.price > 200;

```

f. Count of how many products each customer has bought.

```
SELECT c.cust_name, COUNT(s.item_id) AS products_bought
FROM cust7 c JOIN sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_name;
```

g. List of products bought by customer with cust_id = 5.

```
SELECT i.item_name FROM sale s JOIN item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

h. List item details sold as of today.

```
SELECT DISTINCT i.* FROM item i JOIN sale s ON i.item_id = s.item_id
WHERE TRUNC(s.bill_date) = TRUNC(SYSDATE);
```

i. Create a view listing bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```
CREATE VIEW sale_detail_view AS
SELECT s.bill_no, s.bill_date, s.cust_id, s.item_id, i.price, s.qty_sold,
       (s.qty_sold * i.price) AS amount
FROM sale s JOIN item i ON s.item_id = i.item_id;
```

j. Create a view listing daily sales, date-wise, for the last one week.

```
CREATE VIEW weekly_sales_view AS
SELECT TRUNC(bill_date) AS sale_date, SUM(qty_sold) AS total_qty
FROM sale
WHERE bill_date >= SYSDATE - 7
GROUP BY TRUNC(bill_date);
```

Assignment 8 — PL/SQL Basics

Conditionals, LOOP, WHILE, FOR — anonymous PL/SQL blocks

1. Grade from marks using if-else ladder.

```
DECLARE
    v_marks NUMBER := &marks;
    v_grade VARCHAR2(2);
BEGIN
    IF v_marks >= 90 THEN v_grade := 'A+';
    ELSIF v_marks >= 80 THEN v_grade := 'A';
    ELSIF v_marks >= 70 THEN v_grade := 'B';
    ELSIF v_marks >= 60 THEN v_grade := 'C';
    ELSIF v_marks >= 40 THEN v_grade := 'D';
    ELSE v_grade := 'F';
    END IF;
    DBMS_OUTPUT.PUT_LINE('Grade: ' || v_grade);
END;
/
```

2. Display all even numbers up to a number using a simple LOOP.

```
DECLARE
    v_n NUMBER := &n;
    i NUMBER := 2;
BEGIN
    LOOP
        EXIT WHEN i > v_n;
        DBMS_OUTPUT.PUT_LINE(i);
        i := i + 2;
    END LOOP;
END;
/
```

3. Factorial of a number using FOR loop.

```
DECLARE
    v_n NUMBER := &n;
    v_fact NUMBER := 1;
BEGIN
    FOR i IN 1..v_n LOOP
        v_fact := v_fact * i;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Factorial = ' || v_fact);
END;
/
```

4. Fibonacci series up to 20000 using WHILE loop.

```

DECLARE
    a NUMBER := 0;
    b NUMBER := 1;
    c NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE(a);
    DBMS_OUTPUT.PUT_LINE(b);
    WHILE (a + b) < 20000 LOOP
        c := a + b;
        DBMS_OUTPUT.PUT_LINE(c);
        a := b;
        b := c;
    END LOOP;
END;
/

```

5. Insert random numbers into table random_num (rand_no NUMBER(20)).

```

CREATE TABLE random_num (rand_no NUMBER(20));

DECLARE
BEGIN
    FOR i IN 1..10 LOOP
        INSERT INTO random_num VALUES (TRUNC(DBMS_RANDOM.VALUE(1, 10000000)));
    END LOOP;
    COMMIT;
END;
/

```

6. Fill table sphere(rad, area, volume) with radius values from 1 to 20.

```

CREATE TABLE sphere (rad NUMBER(2), area NUMBER(10,2), volume NUMBER(15,2));

DECLARE
    v_pi CONSTANT NUMBER := 3.1416;
BEGIN
    FOR r IN 1..20 LOOP
        INSERT INTO sphere VALUES (
            r,
            ROUND(4 * v_pi * POWER(r,2), 2),
            ROUND((4/3) * v_pi * POWER(r,3), 2)
        );
    END LOOP;
    COMMIT;
END;
/

```

7. Display name, age and rating from table sailor for a user-input s_id.

```

DECLARE
    v_sid sailor.sid%TYPE := '&sid';
    v_name sailor.sname%TYPE;
    v_age sailor.age%TYPE;
    v_rating sailor.rating%TYPE;
BEGIN
    SELECT sname, age, rating INTO v_name, v_age, v_rating
    FROM sailor WHERE sid = v_sid;
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name || ', Age: ' || v_age || ', Rating: ' || v_rating);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No sailor found with sid ' || v_sid);
END;
/

```

8. (Practice) Display all multiples of 3 up to a number using WHILE loop.

```

DECLARE
    v_n NUMBER := &n;
    i NUMBER := 3;
BEGIN
    WHILE i <= v_n LOOP
        DBMS_OUTPUT.PUT_LINE(i);
        i := i + 3;
    END LOOP;
END;
/

```

9. (Practice) Display multiples of 5 using FOR loop.

```

DECLARE
    v_n NUMBER := &n;
BEGIN
    FOR i IN 1..v_n LOOP
        IF MOD(i,5) = 0 THEN
            DBMS_OUTPUT.PUT_LINE(i);
        END IF;
    END LOOP;
END;
/

```

10. (Practice) Calculate salary from basic pay.

```

DECLARE
    v_basic NUMBER := &basic;
    v_da NUMBER;
    v_hra NUMBER;
    v_salary NUMBER;
BEGIN
    v_da := 0.60 * v_basic;
    v_hra := 0.15 * v_basic;
    v_salary := v_basic + v_da + v_hra;
    DBMS_OUTPUT.PUT_LINE('Gross Salary = ' || v_salary);
END;
/

```

11. (Practice) String reverse.

```

DECLARE
    v_str VARCHAR2(50) := '&input_string';
    v_rev VARCHAR2(50) := '';
BEGIN
    FOR i IN REVERSE 1..LENGTH(v_str) LOOP
        v_rev := v_rev || SUBSTR(v_str, i, 1);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Reversed: ' || v_rev);
END;
/

```

12. (Practice) Find leap year.

```

DECLARE
    v_year NUMBER := &year;
BEGIN
    IF (MOD(v_year,4) = 0 AND MOD(v_year,100) != 0) OR MOD(v_year,400) = 0 THEN
        DBMS_OUTPUT.PUT_LINE(v_year || ' is a Leap Year');
    ELSE
        DBMS_OUTPUT.PUT_LINE(v_year || ' is NOT a Leap Year');
    END IF;
END;
/

```

Assignment 9 — Exception Handling

Predefined exceptions (NO_DATA_FOUND, TOO_MANY_ROWS) and user-defined exceptions

1. Take s_id as input, display record; raise proper predefined exception if not found.

```
DECLARE
    v_sid sailor.sid%TYPE := '&sid';
    v_rec sailor%ROWTYPE;
BEGIN
    SELECT * INTO v_rec FROM sailor WHERE sid = v_sid;
    DBMS_OUTPUT.PUT_LINE(v_rec.sname || ' | ' || v_rec.age || ' | ' || v_rec.rating);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: No sailor exists with sid ' || v_sid);
END;
/
```

2. Take b_name as input, show boat record; raise exception if more than one record matches.

```
DECLARE
    v_bname boat.bname%TYPE := '&bname';
    v_rec boat%ROWTYPE;
BEGIN
    SELECT * INTO v_rec FROM boat WHERE bname = v_bname;
    DBMS_OUTPUT.PUT_LINE(v_rec.bid || ' | ' || v_rec.bname || ' | ' || v_rec.color);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: No boat found with name ' || v_bname);
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Error: More than one boat found with name ' || v_bname);
END;
/
```

3. Take basic as input; compute da, hra, interim, total with user-defined exception logic.

```

DECLARE
    v_basic NUMBER := &basic;
    v_da NUMBER;
    v_hra NUMBER;
    v_interim NUMBER;
    v_total NUMBER;
    e_interim_out_of_range EXCEPTION;
BEGIN
    v_da := 0.60 * v_basic;
    v_hra := 0.15 * v_basic;
    v_interim := 0.35 * (v_basic + v_da);

    IF v_interim BETWEEN 3000 AND 5000 THEN
        v_total := 2 * v_basic;
    ELSIF v_interim > 5000 THEN
        v_total := 3 * v_basic;
    ELSE
        RAISE e_interim_out_of_range;
    END IF;

    DBMS_OUTPUT.PUT_LINE('DA=' || v_da || ' HRA=' || v_hra ||
        ' Interim=' || v_interim || ' Total=' || v_total);
EXCEPTION
    WHEN e_interim_out_of_range THEN
        DBMS_OUTPUT.PUT_LINE('Interim amount (' || v_interim ||
            ') is below 3000 - total cannot be computed by this rule.');
```

END;
/

Assignment 10 — Procedures & Functions

1. Procedure to calculate factorial of a number.

```
CREATE OR REPLACE PROCEDURE proc_factorial(p_n IN NUMBER) IS
    v_fact NUMBER := 1;
BEGIN
    FOR i IN 1..p_n LOOP
        v_fact := v_fact * i;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Factorial of ' || p_n || ' = ' || v_fact);
END;
/
-- call it:
EXEC proc_factorial(5);
```

2. Procedure that takes a user-input date and prints if the year is a leap year.

```
CREATE OR REPLACE PROCEDURE proc_leap_year(p_date IN DATE) IS
    v_year NUMBER := EXTRACT(YEAR FROM p_date);
BEGIN
    IF (MOD(v_year,4) = 0 AND MOD(v_year,100) != 0) OR MOD(v_year,400) = 0 THEN
        DBMS_OUTPUT.PUT_LINE(v_year || ' is a Leap Year');
    ELSE
        DBMS_OUTPUT.PUT_LINE(v_year || ' is NOT a Leap Year');
    END IF;
END;
/
EXEC proc_leap_year(TO_DATE('15-AUG-2024','DD-MON-YYYY'));
```

3. Procedure to find prime number.

```
CREATE OR REPLACE PROCEDURE proc_is_prime(p_n IN NUMBER) IS
    v_is_prime BOOLEAN := TRUE;
BEGIN
    IF p_n < 2 THEN
        v_is_prime := FALSE;
    ELSE
        FOR i IN 2..TRUNC(SQRT(p_n)) LOOP
            IF MOD(p_n, i) = 0 THEN
                v_is_prime := FALSE;
                EXIT;
            END IF;
        END LOOP;
    END IF;

    IF v_is_prime THEN
        DBMS_OUTPUT.PUT_LINE(p_n || ' is Prime');
    ELSE
        DBMS_OUTPUT.PUT_LINE(p_n || ' is NOT Prime');
    END IF;
END;
/
EXEC proc_is_prime(29);
```

4. Procedure to reverse a string.

```

CREATE OR REPLACE PROCEDURE proc_string_reverse(p_str IN VARCHAR2) IS
    v_rev VARCHAR2(100) := '';
BEGIN
    FOR i IN REVERSE 1..LENGTH(p_str) LOOP
        v_rev := v_rev || SUBSTR(p_str, i, 1);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Reversed: ' || v_rev);
END;
/
EXEC proc_string_reverse('DBMS');

```

5. Function that takes a user-input date and returns/prints if the year is a leap year.

```

CREATE OR REPLACE FUNCTION func_leap_year(p_date IN DATE) RETURN VARCHAR2 IS
    v_year NUMBER := EXTRACT(YEAR FROM p_date);
BEGIN
    IF (MOD(v_year,4) = 0 AND MOD(v_year,100) != 0) OR MOD(v_year,400) = 0 THEN
        RETURN v_year || ' is a Leap Year';
    ELSE
        RETURN v_year || ' is NOT a Leap Year';
    END IF;
END;
/
-- call it:
BEGIN
    DBMS_OUTPUT.PUT_LINE(func_leap_year(TO_DATE('01-JAN-2000','DD-MON-YYYY')));
END;
/

```

6. (Practice) Procedure to implement Fibonacci series.

```

CREATE OR REPLACE PROCEDURE proc_fibonacci(p_n IN NUMBER) IS
    a NUMBER := 0;
    b NUMBER := 1;
    c NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE(a);
    DBMS_OUTPUT.PUT_LINE(b);
    FOR i IN 3..p_n LOOP
        c := a + b;
        DBMS_OUTPUT.PUT_LINE(c);
        a := b;
        b := c;
    END LOOP;
END;
/
EXEC proc_fibonacci(10);

```

7. (Practice) Procedure that takes marks and prints the grade.

```

CREATE OR REPLACE PROCEDURE proc_grade(p_marks IN NUMBER) IS
BEGIN
    IF p_marks >= 90 THEN DBMS_OUTPUT.PUT_LINE('A+');
    ELSIF p_marks >= 80 THEN DBMS_OUTPUT.PUT_LINE('A');
    ELSIF p_marks >= 70 THEN DBMS_OUTPUT.PUT_LINE('B');
    ELSIF p_marks >= 60 THEN DBMS_OUTPUT.PUT_LINE('C');
    ELSIF p_marks >= 40 THEN DBMS_OUTPUT.PUT_LINE('D');
    ELSE DBMS_OUTPUT.PUT_LINE('F');
    END IF;
END;
/
EXEC proc_grade(85);

```

8. (Practice) Function to calculate factorial of a number.

```

CREATE OR REPLACE FUNCTION func_factorial(p_n IN NUMBER) RETURN NUMBER IS
    v_fact NUMBER := 1;
BEGIN
    FOR i IN 1..p_n LOOP
        v_fact := v_fact * i;
    END LOOP;
    RETURN v_fact;
END;
/
SELECT func_factorial(6) FROM DUAL;

```

9. (Practice) Function to implement Fibonacci series (returns the nth term).

```

CREATE OR REPLACE FUNCTION func_fibonacci(p_n IN NUMBER) RETURN NUMBER IS
    a NUMBER := 0;
    b NUMBER := 1;
    c NUMBER;
BEGIN
    IF p_n = 1 THEN RETURN a; END IF;
    IF p_n = 2 THEN RETURN b; END IF;
    FOR i IN 3..p_n LOOP
        c := a + b;
        a := b;
        b := c;
    END LOOP;
    RETURN b;
END;
/
SELECT func_fibonacci(8) FROM DUAL;

```

Assignment 11 — Database Triggers

Row-level BEFORE triggers for auditing and validation

1a. Trigger on CUST_MSTR: log previous values on UPDATE/DELETE into AUDIT_CUST.

```
CREATE TABLE cust_mstr (  
    cust_no NUMBER(5) PRIMARY KEY,  
    fname VARCHAR2(15),  
    lname VARCHAR2(15),  
    mname VARCHAR2(15),  
    dob_inc DATE  
);  
  
CREATE TABLE audit_cust (  
    cust_no NUMBER(5),  
    fname VARCHAR2(15),  
    lname VARCHAR2(15),  
    mname VARCHAR2(15),  
    dob_inc DATE,  
    operation VARCHAR2(10),  
    audit_date DATE  
);  
  
CREATE OR REPLACE TRIGGER trg_cust_mstr_audit  
BEFORE UPDATE OR DELETE ON cust_mstr  
FOR EACH ROW  
DECLARE  
    v_operation VARCHAR2(10);  
BEGIN  
    IF UPDATING THEN  
        v_operation := 'UPDATE';  
    ELSIF DELETING THEN  
        v_operation := 'DELETE';  
    END IF;  
  
    INSERT INTO audit_cust  
    VALUES (:OLD.cust_no, :OLD.fname, :OLD.lname, :OLD.mname, :OLD.dob_inc,  
            v_operation, SYSDATE);  
END;  
/
```

1b. Trigger on TRANS_MSTR: validate account no., positive non-zero amount, sufficient balance for withdrawal.

```

CREATE TABLE trans_mstr (
    trans_no NUMBER(6) PRIMARY KEY,
    acct_no NUMBER(6),
    dt DATE,
    type VARCHAR2(10),
    particular VARCHAR2(30),
    dr_cr CHAR(1),
    amt NUMBER(10,2),
    balance NUMBER(10,2)
);

-- assumes an ACCOUNTS master table exists: accounts(acct_no, curr_balance)
CREATE OR REPLACE TRIGGER trg_trans_mstr_validate
BEFORE INSERT ON trans_mstr
FOR EACH ROW
DECLARE
    v_valid_acct NUMBER;
    v_curr_bal NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_valid_acct FROM accounts WHERE acct_no = :NEW.acct_no;
    IF v_valid_acct = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Invalid account number');
    END IF;

    IF :NEW.amt IS NULL OR :NEW.amt <= 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Transaction amount must be positive and non-zero');
    END IF;

    IF :NEW.type = 'WITHDRAWAL' THEN
        SELECT curr_balance INTO v_curr_bal FROM accounts WHERE acct_no = :NEW.acct_no;
        IF :NEW.amt > v_curr_bal THEN
            RAISE_APPLICATION_ERROR(-20003, 'Withdrawal amount exceeds current balance');
        END IF;
    END IF;
END;
/

```

Assignment 12 — Cursors

1. From table reserve, use a cursor to show all boat ids and reservation days for a user-input sailor id.

```
DECLARE
    v_sid sailor.sid%TYPE := '&sid';
    CURSOR c_res IS SELECT bid, r_day FROM reserve WHERE sid = v_sid;
BEGIN
    FOR rec IN c_res LOOP
        DBMS_OUTPUT.PUT_LINE('Boat ID: ' || rec.bid || ' Day: ' || rec.r_day);
    END LOOP;
END;
/
```

2. Display table sailor using a cursor.

```
DECLARE
    CURSOR c_sailor IS SELECT * FROM sailor;
    rec c_sailor%ROWTYPE;
BEGIN
    OPEN c_sailor;
    LOOP
        FETCH c_sailor INTO rec;
        EXIT WHEN c_sailor%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(rec.sid || ' ' || rec.sname || ' ' || rec.rating || ' ' || rec.age);
    END LOOP;
    CLOSE c_sailor;
END;
/
```

3. Use table customer: for a user-input territory, display all other attributes for that territory.

```
DECLARE
    v_territory customer.territory%TYPE := '&territory';
    CURSOR c_cust IS SELECT * FROM customer WHERE territory = v_territory;
BEGIN
    FOR rec IN c_cust LOOP
        DBMS_OUTPUT.PUT_LINE(rec.cust_id || ' ' || rec.cust_fname || ' ' ||
                               rec.cust_lname || ' ' || rec.cred_lmt || ' ' || rec.income);
    END LOOP;
END;
/
```

4. Find all s_id, s_name, and reserved boat for sailors whose age is between 20 and 55.

```
DECLARE
    CURSOR c_age IS
        SELECT s.sid, s.sname, r.bid
        FROM sailor s JOIN reserve r ON s.sid = r.sid
        WHERE s.age > 20 AND s.age < 55;
BEGIN
    FOR rec IN c_age LOOP
        DBMS_OUTPUT.PUT_LINE(rec.sid || ' ' || rec.sname || ' -> Boat ' || rec.bid);
    END LOOP;
END;
/
```

Practice Assignment (Home) — Library Schema

Schema (as given)

Table	Attributes
Student	Stud_no (integer), Stud_name (string)
Membership	Mem_no (integer), Stud_no (integer)
Book	book_no (integer), book_name (string), author (string)
Iss_rec	iss_no (integer), iss_date (date), Mem_no, book_no

a. Create the tables with appropriate integrity constraints.

```
CREATE TABLE student (  
    stud_no NUMBER(4) PRIMARY KEY,  
    stud_name VARCHAR2(20)  
);  
  
CREATE TABLE membership (  
    mem_no NUMBER(4) PRIMARY KEY,  
    stud_no NUMBER(4) REFERENCES student(stud_no)  
);  
  
CREATE TABLE book (  
    book_no NUMBER(4) PRIMARY KEY,  
    book_name VARCHAR2(30),  
    author VARCHAR2(20)  
);  
  
CREATE TABLE iss_rec (  
    iss_no NUMBER(5) PRIMARY KEY,  
    iss_date DATE DEFAULT SYSDATE,  
    mem_no NUMBER(4) REFERENCES membership(mem_no),  
    book_no NUMBER(4) REFERENCES book(book_no)  
);
```

b. Insert around 10 records in each of the tables.

```

INSERT INTO student VALUES (1,'Ankit Das');
INSERT INTO student VALUES (2,'Bristi Roy');
INSERT INTO student VALUES (3,'Chirag Sen');
INSERT INTO student VALUES (4,'Debjani Ghosh');
INSERT INTO student VALUES (5,'Ehsan Ali');
INSERT INTO student VALUES (6,'Falguni Dutta');
INSERT INTO student VALUES (7,'Gaurav Nath');
INSERT INTO student VALUES (8,'Hia Chatterjee');
INSERT INTO student VALUES (9,'Ishaan Kar');
INSERT INTO student VALUES (10,'Jaya Mitra');

INSERT INTO membership VALUES (501,1);
INSERT INTO membership VALUES (502,2);
INSERT INTO membership VALUES (503,3);
INSERT INTO membership VALUES (504,4);
INSERT INTO membership VALUES (505,5);
INSERT INTO membership VALUES (506,6);
INSERT INTO membership VALUES (507,7);
INSERT INTO membership VALUES (508,8);
INSERT INTO membership VALUES (509,9);
INSERT INTO membership VALUES (510,10);

INSERT INTO book VALUES (201,'Database Systems','C J Date');
INSERT INTO book VALUES (202,'Operating Systems','Silberschatz');
INSERT INTO book VALUES (203,'Computer Networks','Tanenbaum');
INSERT INTO book VALUES (204,'Data Structures','Sahni');
INSERT INTO book VALUES (205,'An Intro to DB Systems','C J Date');
INSERT INTO book VALUES (206,'Algorithms','Cormen');
INSERT INTO book VALUES (207,'Software Engineering','Pressman');
INSERT INTO book VALUES (208,'Compiler Design','Aho');
INSERT INTO book VALUES (209,'Digital Logic','Morris Mano');
INSERT INTO book VALUES (210,'Artificial Intelligence','Russell');

INSERT INTO iss_rec VALUES (1,SYSDATE,501,201);
INSERT INTO iss_rec VALUES (2,SYSDATE,502,203);
INSERT INTO iss_rec VALUES (3,SYSDATE,503,206);
INSERT INTO iss_rec VALUES (4,SYSDATE,504,205);
INSERT INTO iss_rec VALUES (5,SYSDATE,505,202);
INSERT INTO iss_rec VALUES (6,SYSDATE,506,208);
INSERT INTO iss_rec VALUES (7,SYSDATE,507,209);
INSERT INTO iss_rec VALUES (8,SYSDATE,508,204);
INSERT INTO iss_rec VALUES (9,SYSDATE,509,207);
INSERT INTO iss_rec VALUES (10,SYSDATE,510,210);

```

c. List all student names with their membership numbers.

```

SELECT s.stud_name, m.mem_no
FROM student s JOIN membership m ON s.stud_no = m.stud_no;

```

d. List all issues for the current date with student and book names.

```

SELECT s.stud_name, b.book_name, i.iss_date
FROM iss_rec i
JOIN membership m ON i.mem_no = m.mem_no
JOIN student s ON m.stud_no = s.stud_no
JOIN book b ON i.book_no = b.book_no
WHERE TRUNC(i.iss_date) = TRUNC(SYSDATE);

```

e. List details of students who borrowed a book whose author is C J Date.

```
SELECT DISTINCT s.* FROM student s
JOIN membership m ON s.stud_no = m.stud_no
JOIN iss_rec i ON m.mem_no = i.mem_no
JOIN book b ON i.book_no = b.book_no
WHERE b.author = 'C J Date';
```

f. Count of how many books each student has borrowed.

```
SELECT s.stud_name, COUNT(i.book_no) AS books_borrowed
FROM student s
JOIN membership m ON s.stud_no = m.stud_no
JOIN iss_rec i ON m.mem_no = i.mem_no
GROUP BY s.stud_name;
```

g. List of books taken by student with stud_no = 5.

```
SELECT b.book_name
FROM book b JOIN iss_rec i ON b.book_no = i.book_no
JOIN membership m ON i.mem_no = m.mem_no
WHERE m.stud_no = 5;
```

h. List book details issued as of today.

```
SELECT DISTINCT b.* FROM book b JOIN iss_rec i ON b.book_no = i.book_no
WHERE TRUNC(i.iss_date) = TRUNC(SYSDATE);
```

i. Create a view listing iss_no, iss_date, stud_name, book_name.

```
CREATE VIEW iss_view AS
SELECT i.iss_no, i.iss_date, s.stud_name, b.book_name
FROM iss_rec i
JOIN membership m ON i.mem_no = m.mem_no
JOIN student s ON m.stud_no = s.stud_no
JOIN book b ON i.book_no = b.book_no;
```

j. Create a view listing daily issues, date-wise, for the last one week.

```
CREATE VIEW weekly_issues_view AS
SELECT TRUNC(iss_date) AS issue_date, COUNT(*) AS total_issues
FROM iss_rec
WHERE iss_date >= SYSDATE - 7
GROUP BY TRUNC(iss_date);
```